

Graph Representation Learning for Microarchitecture Design Space Exploration

Xiaoling Yi^{1,2}, Jialin Lu^{1,2}, Xiankui Xiong^{4,5}, Dong Xu^{4,5}, Li Shang^{1,3}, Fan Yang^{1,2*}

¹State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China

²School of Microelectronics, Fudan University, Shanghai, China

³China and Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University, Shanghai, China

⁴ZTE Corporation, China

⁵State Key Laboratory of Mobile Network and Mobile Multimedia Technology, China

Abstract—Design optimization of modern microprocessors is a complex task due to the exponential growth of the design space. This work presents GRL-DSE, an automatic microarchitecture search framework based on graph embeddings. GRL-DSE uses graph representation learning to build a compact and continuous embedding space. Multi-objective Bayesian optimization using an ensemble surrogate model conducts microarchitecture design space exploration in the graph embedding space to efficiently and holistically optimize performance-power-area (PPA) objectives. Experimental studies on RISC-V BOOM show that GRL-DSE outperforms previous techniques by 74.59% on Pareto front quality and outperforms manual designs in terms of PPA.

I. INTRODUCTION

Microarchitecture design space exploration (DSE) is a challenging problem due to the combinatorial explosion of the search space and costly evaluation. For decades, the microarchitecture design space of modern microprocessors has continued to expand with the addition of performance-enhancing features, such as branch predictors, I/D Caches, special queues, and buffers, to name a few. For instance, the RISC-V BOOM design space can be as large as 1.6×10^8 [1], and the design space of high-performance multi-core architecture can be several orders of magnitude larger. Furthermore, microarchitecture simulations using standard benchmark suites, the de facto performance evaluation method, are time consuming. Therefore, an exhaustive traversal and simulation of each microarchitecture design point is simply not feasible.

Statistical and machine learning based methods have recently been introduced for efficient processor microarchitecture modeling [1]–[8]. These methods often treat the processor microarchitecture as a black box, and leverage a training dataset, e.g., via processor simulations, to train a surrogate microarchitecture model. Once trained, the surrogate model can support efficient performance prediction, e.g., thousands of predictions in a few seconds. Sitting the predictive model in the inner loop of the optimization flow provides a promising opportunity for fast microarchitecture design space exploration. Nonetheless, optimization algorithms still need to search against an exponentially exploding design space, making scalable microarchitecture design space exploration a very challenging goal.

We argue that incorporating microarchitecture domain knowledge, e.g., microarchitecture organization and parameter dependencies, into processor modeling is critical for enabling efficient microarchitecture design space exploration. In fact, microprocessor architecture design is a highly structured exploration process, constrained by a series of microarchitecture-specific structures and dependencies, such as data and control dependencies governed by the microprocessor pipeline structure. Therefore, key microarchitectural parameters are closely related to each other with tight constraints. For example, the instruction *decode width* must divide the *re-order buffer entry number* and the instruction *fetch width* should be larger than the instruction *decode width*. These microarchitecture intrinsic features and design constraints can effectively reduce the exploration cost. A prior study in microprocessor design space exploration [9] uses microarchitecture information, i.e., the relationships between parameters, to avoid unnecessary simulations of design points, providing more than one order of magnitude reduction in the number of simulations needed for exploration. However, the domain knowledge that indicates the

processor microarchitecture information is ignored in existing statistical and machine learning based models. Such domain knowledge is heuristic, causing it difficult to formulate and capture. Treating the processor microarchitecture as a black box is easy to manipulate, but requires a large amount of training data to ensure sufficient inference accuracy, because the surrogate model has no prior knowledge of the microprocessor behavior, and it learns from scratch. For instance, [2] requires 2.5K training data to build a predictive model for multiprocessors. In addition, the black-box surrogate model lacks necessary microarchitecture insights to guide optimization algorithms to search the design space efficiently. Recently, BOOM-Explorer [1] proposes an initial sampling algorithm that explicitly divides the entire design space into several subspaces according to the microarchitectural key parameter *decode width* and then chooses representative design points in each subspace, effectively reducing the required initial number of samples for the surrogate model training. However, applying this approach to practical design space exploration is challenging because the domain knowledge of the key parameter *decode width* is obtained through exhaustive analysis of an offline simulated dataset. Such offline simulation dataset is expensive to acquire and application specific. Nonetheless, this work [1] demonstrates the great potential of embedding domain knowledge into the design space exploration process.

This paper presents GRL-DSE, an automatic microarchitecture search framework to enable efficient design space exploration. As shown in Figure 1, GRL-DSE consists of the following two components.

- First, we propose a new directed acyclic graph (DAG) [10] model to represent the processor microarchitecture hardware organization and parameter relationships. The microarchitecture DAGs that represent different microprocessor configurations are then fed into a customized graph neural network to extract and transform key microarchitecture features as embedding vectors in a compact and continuous graph embedding space. Yield graph representations comprise inherent microarchitecture domain knowledge, i.e., intrinsic microarchitecture organization and parameter dependencies. To the best of our knowledge, this is the first work that employs graph representation learning (GRL) to capture the intrinsic characteristics of the microprocessors to enable efficient microarchitecture design space exploration.
- Second, we propose a multi-objective Bayesian optimization (BO) method to conduct microarchitecture design space exploration directly in the yield graph embedding space to efficiently and holistically optimize the performance-power-area (PPA) objectives. The multi-objective BO is equipped with a surrogate model that is trained to learn the mapping between the graph embedding space and the microprocessor PPA objective space. We propose a robust ensemble learning strategy in the surrogate model to reduce the predictive uncertainty.

Experimental studies on RISC-V BOOM demonstrate that GRL-DSE outperforms previous arts by up to 74.59% on Pareto front quality in terms of generational distance. The design points produced by GRL-DSE outperform manual designs developed by senior engineers 0.11%-3.26% in terms of PPA. Besides, the proposed GRL-

*Corresponding author: yangfan@fudan.edu.cn.

DSE achieves the best performance compared with the winners at the ICCAD CAD Contest 2022.

The rest of this paper is organized as follows. § II presents the RISC-V BOOM microarchitecture overview and problem formulation. § III describes the proposed method. § IV presents the experimental results. § V concludes the work.

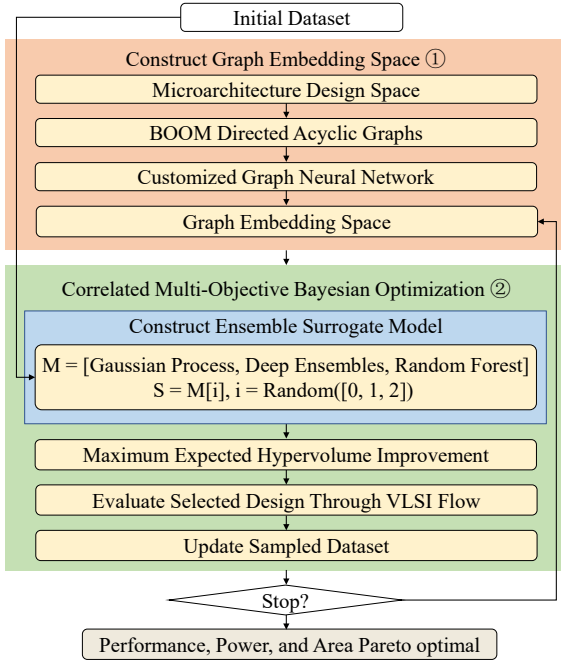


Fig. 1: GRL-DSE framework overview.

II. BACKGROUND

A. RISC-V BOOM

As an open-source instruction set architecture (ISA), RISC-V has attracted growing attention among academia and industry in recent years. The Berkeley Out-of-Order Machine (BOOM) [11], [12] is an out-of-order superscalar RISC-V processor with industry-competitive PPA among embedded low-power cores. Benefited from the flexible parameterization capability provided by Chisel [13], the BOOM microarchitecture can be conveniently customized with different parameter configurations, i.e., *FetchWidth* and *DecodeWidth*, to meet various design specifications. Key microarchitectural parameters are summarized in TABLE I. Each row in TABLE I specifies possible configurations of the corresponding functional module, which defines the BOOM microarchitecture design space. In this work, we use BOOM as the testbed to evaluate the effectiveness of the proposed DSE framework.

B. Problem Formulation

This work focuses on microarchitecture design space exploration for modern microprocessors. Given a microprocessor architecture, e.g., RISC-V, and the corresponding microarchitecture design space, e.g., parametric configurations in vector form, the goal is to search the design space and determine the Pareto optimal front in terms of PPA, specifically:

Definition 1 (Pareto Optimality). For a m -dimension minimization problem, a solution x_1 is said to dominate a solution x_2 if $\forall i \in \{1, \dots, m\}, f_i(x_1) \leq f_i(x_2)$ and $\exists j \in \{1, \dots, m\}, f_j(x_1) < f_j(x_2)$, then we denote $x_1 \succeq x_2$. The set of designs that are not dominated by any others is called the Pareto-optimal set and forms the Pareto optimality in the design space. The set of Pareto-optimal outcomes is called the Pareto front.

TABLE I: Microarchitecture design space of BOOM. The definition of these parameters can refer to [11].

Module	Parameters	Values
Front End	FetchWidth	4, 8
	FetchBufferEntry	8, 16, 24, 32, 35, 40
	FetchTargetQueueEntry	16, 40, 32
	ICacheSets	32, 64, 128
	ICacheWays	4, 8
	ICacheTLBSets	1, 2
	ICacheTLBWays	8, 16, 32
	BranchCount	8, 6, 10, 12, 10, 14, 16, 14, 18, 20, 22, 24, 20, 26
	DecodeWidth	1, 2, 3, 4, 5
	RobEntries	32, 64, 96, 128, 130
Execution Core	Int/FpPhysRegisters	48, 64, 80, 96, 112
	Mem/Fp/IntDispatchWidth	1, 2, 3, 4, 5
	Mem/Fp/IntNumEntries	4, 8, 12, 16, 20, 24, 32, 40
	Mem/FpIssueWidth	1, 2
	IntIssueWidth	1, 2, 3, 4, 5
	Ld/StqEntries	8, 16, 24, 32
	DCacheSets	32, 64, 128
	DCacheWays	2, 4, 8
	DCacheTLBSets	1, 2
	DCacheTLBWays	8, 16, 32
	DCacheMSHR	2, 4, 8
	ReplacementPolicy	0, 1

Problem 1 (Microarchitecture Design Space Exploration). Given a microarchitecture design space $\mathcal{D}$, each microarchitecture configuration inside $\mathcal{D}$ is regarded as a feature vector x and the corresponding PPA metrics y forms the objective space $\mathcal{Y}$. For each x , the corresponding PPA metrics $y \in \mathcal{Y}$, which can be estimated through the VLSI implementation flow f . Processor microarchitecture DSE is then defined as finding a set of microarchitectures X that form the Pareto optimality in terms of $Y \in \mathcal{Y}$. Hence, $X = \{x | x' \not\succeq x, \forall x' \in \mathcal{D}\}$, $Y = \{f(x) | x \in X, \forall x \in \mathcal{D}\}$.

III. PROPOSED APPROACH

This section presents the proposed GRL-DSE framework. As shown in Figure 1, in GRL-DSE, a microprocessor is modeled as a microarchitecture DAG (§ III-A), which is then transformed into a continuous graph embedding space using a customized graph neural network model (§ III-B). The proposed Bayesian optimization algorithm equipped with an ensemble surrogate model conducts design space exploration directly on the continuous graph embedding space and searches Pareto-optimal microarchitecture configurations (§ III-C).

A. Microarchitecture Graph Model

In GRL-DSE, the microarchitecture of an out-of-order superscalar microprocessor is modeled as a DAG. Specifically, as shown in Figure 2, using DAG, the superscalar microarchitecture is modeled as two subgraphs, namely Front End and Execution Core. The Front End (producer), consisting of two graph nodes IF (instruction fetch) and ID (instruction decode), models instruction fetch, decoding, and dispatch. The Execution Core (consumer) consists of five graph nodes, PRF (physical register file), INT (integer-related instruction execution units), FP (floating point-related instruction execution units), MEM (memory-related instruction execution units), and LSU (load-store unit), orchestrating the execution of the translated micro-operations and results write-back. In the microarchitecture DAG, key microarchitecture parameters are embedded as node features. For illustration purposes, Table II lists key microarchitecture features of BOOM encoded in each graph node. The microarchitecture DAG can be extended to incorporate other microarchitecture features or support other disparate processors.

The microarchitecture DAG captures the instruction execution process, i.e., flowing through the microprocessor pipeline. Specifically, the microprocessor instruction stream is modeled using directed edges. Each directed edge represents a specific producer-consumer relationship between microarchitecture components. For example, the edge from *IF* to *ID* indicates instructions flowing from the fetch

phase to the decode phase. The edges from *ID* to *INT*, *FP*, and *MEM* execution units model instruction dispatch operation. In addition, the directed edges also encode parameter dependency relationships. To minimize pipeline stalls, the throughput of downstream components (modeled with corresponding microarchitecture parameters) should match that of the upstream components correspondingly, which introduces microarchitecture dependencies between downstream and upstream components. For instance, *FetchBufferEntry* must be larger than *FetchWidth*, and the *DecodeWidth* should equal to or smaller than *FetchWidth*, whereas *DecodeWidth* should divide *RobEntries* and *FetchBufferEntry*.

In summary, the proposed DAG contains a rich set of microarchitecture information, such as microarchitecture configurations, inter-component data and control dependencies as well as parameter correlations.

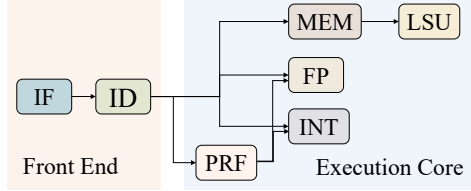


Fig. 2: The microarchitecture DAG of out-of-order superscalar microprocessors, consisting of seven graph nodes with node features encoding microarchitecture configurations and directed edges indicating instruction execution orders or parametric dependencies.

TABLE II: Microarchitecture parameters encoded in node features.

Node	Node feature
<i>IF</i>	FetchWidth, FetchBufferEntries, BranchTag, FetchTargetQueueEntry,
<i>ID</i>	ICacheSets, ICacheWays, ICacheTLBSets, ICacheTLBWays
<i>PRF</i>	DecodeWidth, ROBEntries
<i>INT</i>	IntPhysRegisters, FpPhysRegisters
<i>FP</i>	IntDispatchWidth, IntIssueWidth, IntNumEntries
<i>MEM</i>	FpDispatchWidth, FpIssueWidth, FpNumEntries
<i>LSU</i>	MemDispatchWidth, MemIssueWidth, MemNumEntries
	LDQEntries, STQEntries, DCacheSets, DCacheWays,
	ReplacementPolicy, MSHR, DCacheTLBSets, DCacheTLBWays

B. Customized Graph Neural Network Model

The microarchitecture DAG is in a discrete representation space and difficult to be directly optimized. Motivated by recent work in graph neural network (GNN) [10], [14], we propose a customized graph neural network model to transform the microarchitecture DAG representation into a continuous graph embedding space to support efficient design space exploration.

Algorithm 1 Graph Representation Learning Algorithm in GRL-DSE

```

1: INPUT: DAG that represents the microarchitecture configurations  $G = (\mathcal{V}, \mathcal{E})$ ;
2: OUTPUT: Embedding vector  $z$  of  $G$  in the continuous graph embedding space;
3: for  $v$  in  $\mathcal{V}$  do
4:   get  $v$ 's predecessors  $v_p$ ;
5:    $\mathbf{h}_v^{in} = \mathcal{A}(\mathbf{h}_{v_p} : v_p \rightarrow v)$ ;
6:    $\mathbf{h}_v = \mathcal{U}(V_v, \mathbf{h}_v^{in})$ ;
7: end for
8:  $q(\mathbf{Z}|G) = \text{MLP}(\mathbf{h}_v)$ ;
9: sample  $z$  from  $q(\mathbf{Z}|G)$ ;
10: return  $z$ ;

```

Graph Neural Network Architecture. Algorithm 1 summarizes the proposed GNN-based graph representation learning process. It uses asynchronous information propagation to encode the computations on the microarchitecture DAG. Specifically, the graph nodes are arranged according to the topological order of the directed edges, and

hidden states of the nodes are updated accordingly, which conforms to microprocessor instruction execution flow and parameter dependencies. Different from [10], our GNN-based method simultaneously considers both node and edge features. As shown in Algorithm 1, similar to the standard graph convolutional network, the update function $\mathcal{U}$ and the aggregation function $\mathcal{A}$ are both used to obtain the hidden state $\mathbf{h}_v$ of the graph nodes. For every node v in the DAG, $\mathbf{h}_v^{in}$ is the aggregation of the states of all v 's predecessors. Figure 3 ① illustrates how the instruction execution and inter-node parameter dependencies PF are propagated. Since nodes have different number of features, a front-end linear transformation layer is introduced to ensure the same node feature size across hidden state update and aggregation. Specifically, $\mathcal{A}$ and $\mathcal{U}$ are described as follows:

$$\mathcal{A}(\mathbf{h}_{v_p} : v_p \rightarrow v) = \sum_{v_p} g(V'_{v_p} \times \mathbf{h}_{v_p}) \odot m(V'_{v_p} \times \mathbf{h}_{v_p}), \quad (1)$$

$$\mathcal{U}(V_v, \mathbf{h}_v^{in}) = \text{GRU}(W_v V_v, \mathbf{h}_v^{in}), \quad (2)$$

where g is a gating network, m is a mapping network, and $\odot$ is Hadamard product. V_v represents the node feature of node v , and V_{v_p} represents the node feature of its predecessors v_p . $V'_{v_p} = W_{v_p} V_{v_p}$, where W_{v_p} is the linear transformation layer of v_p . A gated recurrent unit (GRU) [15] is used to model the update function $\mathcal{U}$.

As illustrated in Figure 3 ②, a virtual input node and an output node are added in the DAG as the unique start and end point. The virtual output node is the successor of node *INT*, *FP*, and *LSU*. The information propagation in the microarchitecture DAG starts from the virtual input node V_{in} and ends with a virtual node V_{out} . The hidden state at the last node $\mathbf{h}_{V_{out}}$ can be viewed as a microarchitecture representation that extracts the key processor microarchitecture information. The microarchitecture representation is then fed to two MultiLayer-Perceptrons (MLP) to produce the microarchitecture graph embedding vector z .

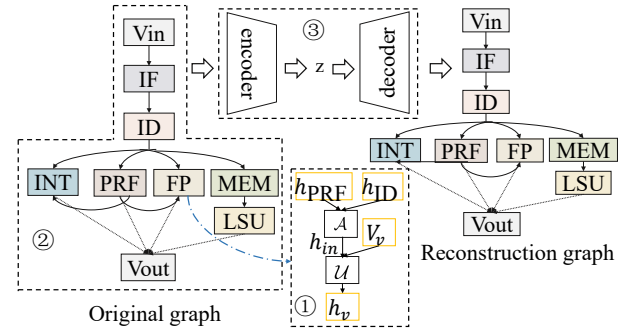


Fig. 3: Microarchitecture DAG information propagation mechanism and VGAE training methodology illustration. We change nodes' relative position in the microarchitecture DAG compared with Figure 2 for better illustration.

Training. To train the customized GNN model, we follow the variational graph auto-encoder (VGAE) [16] framework as shown in Figure 3 ③. The GNN model acts as an encoder. The training process, which is illustrated in Figure 3, minimizes the discrepancies between the input and output DAGs by maximizing the variational low bound $\mathcal{L}$ concerning the variational parameters $\mathbf{W}_i$:

$$\mathcal{L}(\theta, \phi; X) = \mathbb{E}_{z \sim q_\theta(\mathbf{Z}|X)} [\log p_\phi(\mathbf{X}|\mathbf{Z})] - KL[q_\theta(\mathbf{Z}|X) || p(\mathbf{Z})], \quad (3)$$

where $q_\theta(\mathbf{Z}|X)$ is an approximated posterior distribution (the encoder) and $p_\phi(\mathbf{X}|\mathbf{Z})$ is a probabilistic generative model (the decoder). θ and ϕ are parameters of the encoder and decoder respectively. $KL[q(\cdot)||p(\cdot)]$ represents the Kullback-Leibler divergence between $q(\cdot)$ and $p(\cdot)$. We take a Gaussian prior $p(\mathbf{Z}) = \prod_i p(z_i) = \prod_i \mathcal{N}(0, \mathbf{I})$. This ensures that the graph embedding vector z follows the Gaussian distribution.

For the VGAE decoder, we use another GRU_{decode} to update the embedded vector and then feed it to a MLP_{decode} to compute the node feature probability $P_{nodefeature}$. After that, based on the probability $P_{nodefeature}$, we can sample the reconstructed nodes $\mathcal{V}_{re}$, and obtain the reconstructed DAG $G_{re} = (\mathcal{V}_{re}, \mathcal{E})$.

C. Design Space Exploration in the Graph Embedding Space

The trained GNN-based graph embeddings form a continuous latent space (graph embedding space $\mathcal{G}$), representing the processor microarchitectures design space (matrix $\mathbf{Z}$). We propose a Bayesian optimization (BO) [17] based method to conduct efficient design space exploration in the continuous graph embedding space ($\mathcal{G}$). It employs a probabilistic online surrogate model in conjunction with an acquisition function to navigate the trade-offs between exploration (evaluating designs with high uncertainty) and exploitation (evaluating designs with high quality). Leveraging an accurate surrogate model and a dedicated acquisition function, BO is sample-efficient for identifying optimized trade-offs between the PPA objectives. The proposed BO algorithm flow is shown in Figure 1 ②. Next, the proposed ensemble surrogate model and EHVI acquisition function are explained in detail.

1) **Ensemble Surrogate Model:** Single machine learning regression model based prediction model, i.e., PPA prediction, may suffer from high prediction variabilities under different benchmarks, and thus mislead decision making [18]. Motivated by the recent progress on ensemble learning [19], we propose a heuristic ensemble surrogate model method for robust and accurate characterization of the relationship between graph embedding space $\mathcal{G}$ and PPA objective space $\mathcal{Y}$. Specifically, we ensemble the Gaussian Process [20], Deep Ensembles [21] and Random Forest [22] as a combinational surrogate model $\mathcal{M}$. At each BO iteration, we randomly select one surrogate $\mathcal{S}$ from $\mathcal{M}$ for PPA prediction. The formulation of the proposed ensemble surrogate model strategy is summarized as follows.

Let training dataset $D = \{X, \mathcal{Y}\}$ consist of N i.i.d. data points, where $X = \{x_1, \dots, x_N\}$ and $\mathcal{Y} = \{f(x_1), \dots, f(x_N)\}$. Given a new point x_* , the posterior distribution $p(y_*|x_*)$ can be modeled as a Gaussian distribution $\mathcal{N}(\mu_{y_*|y}, \sigma_{y_*|y}^2)$.

Using Gaussian Process model, we have:

$$\begin{cases} \mu_{1y_*|y} = k(x_*, X)K^{-1}\mathcal{Y}, \\ \sigma_{1y_*|y}^2 = k(x_*, x_*) - k(x_*, X)K^{-1}k(X, x_*), \end{cases} \quad (4)$$

where kernel function $k(\cdot, \cdot) = \text{Linear kernel} + \text{Matérn } 3/2 \text{ kernel}$, $K = k(X, X) + \sigma_n^2 I$, observation noise $\epsilon_n \sim \mathcal{N}(0, \sigma_n^2)$.

Using Deep Ensembles model, we have:

$$\begin{cases} \mu_{2y_*|y} = \frac{\sum_{m=1}^M \mu_{\theta_m}(x_*)}{M}, \\ \sigma_{2y_*|y}^2 = \frac{\sum_{m=1}^M (\sigma_{\theta_m}^2(x_*) + \mu_{\theta_m}^2(x_*))}{M} - \mu_{2y_*|y}^2, \end{cases} \quad (5)$$

where M is the number of NNs in the ensemble, $\{\theta_m\}_{m=1}^M$ is the parameters of the ensemble, m th NN $\sim \mathcal{N}(\mu_{\theta_m}, \sigma_{\theta_m}^2)$.

Using Random Forest model, we have:

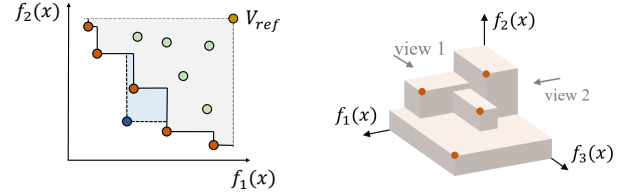
$$\begin{cases} \mu_{3y_*|y} = \frac{1}{M} \sum_{m=1}^M DT_m(x_*), \\ \sigma_{3y_*|y}^2 = \frac{\sum_{m=1}^M (DT_m(x_*) - \mu_{3y_*|y})^2}{M - 1}, \end{cases} \quad (6)$$

where M is the number of Decision Trees, DT_m denotes the m th Decision Tree model.

All three models are trained/updated using the whole sampled dataset. We use $\mathcal{S}$ in Equation 7 as a final surrogate model at each BO iteration to predict x_* 's correlated PPA metrics.

$$\begin{cases} \mathcal{M} = \left[\mathcal{N}\left(\frac{\mu_1}{\sigma_1^2}\right), \mathcal{N}\left(\frac{\mu_2}{\sigma_2^2}\right), \mathcal{N}\left(\frac{\mu_3}{\sigma_3^2}\right) \right], \\ \mathcal{S} = \mathcal{M}[i], i = \text{Random}([0, 1, 2]). \end{cases} \quad (7)$$

2) **EHVI Acquisition Function:** The proposed method uses Expected Hypervolume Improvement (EHVI) [23], which is commonly used for multi-objective compromise, as the acquisition function to guide the search process and characterize the trade-offs between PPA efficiently. The 2-dimensional (2-D) and 3-dimensional (3-D) hypervolume are illustrated in Figure 4. As shown in Figure 4(a) gray region, given a reference point $r \in \mathbb{R}^M$, the hypervolume (HV) of a finite approximate Pareto frontier $\mathcal{P}$ is the M-dimensional *Lebesgue measure* λ_M of the space dominated by $\mathcal{P}$ and bounded from below by r : $HV(\mathcal{P}|r) = \lambda_M(\cup_{i=1}^{|\mathcal{P}|} [r, y_i])$, where $[r, y_i]$ denotes the hyper-rectangle bounded by vertices r and y_i . As shown in Figure 4(a) blue region, the hypervolume improvement (HVI) of a set of points $\mathcal{P}'$ is $HVI(\mathcal{P}'|\mathcal{P}, r) = HV(\mathcal{P} \cup \mathcal{P}'|r) - HV(\mathcal{P}|r)$.



(a) Hypervolume of 2-D objective. (b) Hypervolume of 3-D objective.

Fig. 4: Hypervolume illustration. (a) The region covered in gray is the hypervolume that is dominated by the currently explored Pareto-optimal set denoted as circles in orange. Circles in green denote dominated designs. The circle in blue denotes an explored potential candidate of the Pareto-optimal set among the entire design space. The corresponding HVI is represented as the area of the sub-region colored in light blue. (b) The reference point is the original point hidden behind the cube in this figure.

EHVI is the expectation of HVI over the posterior $P(f(x)|D)$: $\alpha_{EHVI}(\mathcal{X}_{cand}|\mathcal{P}) = \mathbb{E}[HVI(f(\mathcal{X}_{cand})|\mathcal{P})]$. In our method, EHVI is approximated with the commonly used Monte Carlo integration [24], which is given by:

$$\begin{aligned} \alpha_{EHVI}(\mathcal{X}_{cand}|\mathcal{P}) &\approx \hat{\alpha}_{EHVI}(\mathcal{X}_{cand}|\mathcal{P}) \\ &= \frac{1}{N} \sum_{t=1}^N HVI(\tilde{f}_t(\mathcal{X}_{cand})|\mathcal{P}), \end{aligned} \quad (8)$$

where $\tilde{f}_t \sim P(f(x)|D)$ for $t = \{1, \dots, N\}$ is given by the final surrogate model $\mathcal{S}$. In each BO iteration, we traverse each design point and choose $\mathcal{X}_{cand}$ that maximizes the EHVI. It ensures that the new approximate Pareto front $\mathcal{P} \cup \mathcal{X}_{cand}$ has a maximum expected improvement in maximizing instruction execution speed and minimizing energy as well as area expansion. Using EHVI as acquisition function to guide the search process, PPA can be optimized efficiently and holistically.

IV. EXPERIMENTAL RESULTS

This section evaluates the proposed GRL-DSE by comparing it with the state-of-the-art processor microarchitecture DSE algorithm, the winners of the ICCAD CAD Contest 2022, and manual designs.

A. Design Space Exploration Experiment Setup

We construct a design space $\mathcal{D}$ consisting of 1141 data points that are sampled randomly and uniformly from the BOOM design space as referred to in TABLE I. Design space exploration experiments are conducted on this design space $\mathcal{D}$. We utilize 7-nm ASAP7 PDK [25] for the VLSI implementation flow. A customized back-end automatic VLSI implementation flow script is implemented using the Chipyard [26] framework as the front end to generate the Verilog for various BOOM designs. Synopsys VCS P-2019.06-SP2 is used to simulate the design running at 1GHz. Design Compiler Q-2019.12-SP5 is used for power and area metric estimation. Diverse benchmarks are selected to evaluate the performance of microarchitectures,

i.e., dhrystone, median, mt-matmul, mt-vvadd, multiply, pmp, qsort, rsort, spmv, towers, vvadd, and mm. These benchmarks are selected from commonly used CPU benchmark suites and provide good coverage of RISC-V instructions. The VLSI implementation flow runs on an Intel Xeon Platinum 8354H CPU@3.10GHz cluster, and normally takes approximately 3-6 hours for each design. Performance, power, and area values are normalized using a z-score normalization method to remove the numerical imbalance.

The state-of-the-art BOOM DSE method BOOM-Explorer [1], which won ICCAD 2021 Best Paper, is compared with GRL-DSE as the baseline. For a fair comparison, the surrogate model is initialized with 10 microarchitectures sampled from design space $\mathcal{D}$ using the MicroAL algorithm [1], followed by 10 rounds of Bayesian exploration, for all algorithms. All experiments are repeated 10 times along with the baseline, and the corresponding average results are reported.

The algorithm evaluation objective is based on generational distance (GD) [27] and hypervolume (HV), which can comprehensively measure the quality of the resulting approximation Pareto front. GD, as shown in Equation 9, is a commonly used indicator in DSE problems to measure how close the learned Pareto-optimal set is to the actual Pareto-optimal set.

$$GD(\Gamma, \Omega) = \frac{1}{|\Gamma|} \sum_{\gamma \in \Gamma} \min_{\omega \in \Omega} L(\gamma, \omega), \quad (9)$$

where Γ is the learned Pareto front and Ω is the golden Pareto front. L is l_2 -norm. We evaluate each design point in the design space $\mathcal{D}$ to obtain the real Pareto-optimal set and the golden Pareto front.

Besides, ICCAD CAD Contest 2022 [28] provides an evaluation platform that comprises 15633 BOOM design points and corresponding PPA. After submitting the algorithm, the platform will give back an overall score concerning hypervolume and overall running time (algorithm running time and the VLSI implementation flow time). We evaluate the proposed GRL-DSE in that platform and compare the overall score of GRL-DSE with the contest winners.

B. Graph Embedding Space Evaluation

1) *Graph Neural Network Training*: We generate an appropriately 15,000 DAGs dataset. Each DAG represents a valid BOOM with different microarchitecture configurations. As typical unsupervised learning, the training of our GNN does not need to execute any time consuming VLSI implementation flow. The teacher forcing [29] method is used to measure the reconstruction loss following [10]. Mini-batch gradient descent is used to optimize the VGAE loss. A GeForce RTX 2080 Ti GPU is used to train the model for 500 epochs, and the entire training is completed within 10 hours. Note that training only needs to do once. The trained model can be reused for BOOM optimization, such as targeting different multi-objective design specifications, e.g., considering reliability and manufacturability.

2) *Graph Embedding Space Quality*: To evaluate the quality of the graph embedding space, we conduct a reverse mapping experiment. We randomly choose three points in the PPA objective space $\mathcal{Y}$. y_1 and y_3 are DAGs randomly selected, and y_2 is close to y_1 . The three points in objective space $\mathcal{Y}$ are mapped back in the original parameter vector space, denoted as x_1, x_2 and x_3 and graph embedding space, denoted as z_1, z_2 and z_3 , respectively. Then the normalized distance between these points is calculated. We repeat this experiment 10 times and report average results as listed in Table III. The distance between x_1, x_2 is 85.06% larger than the distance between z_1, z_2 , which demonstrates the continuity of the proposed graph embedding space.

TABLE III: Normalized distance comparison in parameter vector space and graph embedding space.

Design space	Parameter vector space		Graph embedding space	
	x_1, x_2	x_1, x_3	z_1, z_2	z_1, z_3
Normalized distance	0.161	0.839	0.087	0.913

C. BOOM Design Space Exploration

TABLE IV(a) shows the GD and hypervolume comparison of GRL-DSE and BOOM-Explorer after conducting design space exploration at our local platform. As we can see, GRL-DSE outperforms the BOOM-Explorer by 74.59% in GD and 1.24% in hypervolume. To further evaluate the efficacy of graph representation learning, we use bare BO to conduct a comparative experiment. GRL-DSE without graph representation learning only obtains 19.92% improvement in GD and 0.73% more hypervolume than BOOM-Explorer. The degraded GD and hypervolume metrics demonstrate the efficacy of graph representation learning. TABLE IV(b) shows the GRL-DSE evaluation result at ICCAD CAD Contest 2022 platform. The proposed GRL-DSE achieves the best performance compared with the contest winners.

Figure 5 shows the learned Pareto front obtained by the GRL-DSE and BOOM-Explorer. From Figure 5, one can intuitively find that GRL-DSE outperforms BOOM-Explorer remarkably. The hypervolume that is dominated by the Pareto front found by GRL-DSE and bound from below by the reference point is larger. The Pareto front learned by GRL-DSE is also much closer to the real Pareto front. The overall declined GD indicates GRL-DSE explores the design space more effectively and comprehensively, providing the designer with higher-quality BOOM microarchitectures for different applications.

TABLE IV: BOOM design space exploration results. (a) GD and hypervolume metrics comparison of GRL-DSE and BOOM-Explorer at the local platform. (b) Scores of GRL-DSE and the winners at ICCAD CAD Contest 2022 platform. Larger scores are better.

(a) Local platform			(b) ICCAD platform	
Methodologies	GD	HV	Winners	Score
<i>Golden</i>	0	19.348	<i>Top 1</i>	52.09
<i>BOOM-Explorer</i>	0.325	17.985	<i>Top 2</i>	51.40
<i>GRL-DSE w/o GRL</i>	0.271	18.117	<i>Top 3</i>	51.13
<i>GRL-DSE</i>	0.186	18.206	<i>GRL-DSE</i>	52.22

D. Optimal BOOM Microarchitecture and Manual Design Analysis

TABLE V: Microarchitectural parameters of two dominated Pareto-optimal designs. The parameters are in the same order as TABLE I.

Microarchitecture	Design Parameters
<i>Pareto Design 1</i>	[4, 8, 16, 32, 4, 1, 16, 8, 1, 34, 52, 48, 1, 1, 1, 6, 6, 6, 1, 1, 1, 12, 12, 64, 4, 1, 8, 2, 1]
<i>Pareto Design 2</i>	[4, 16, 32, 32, 4, 1, 16, 12, 2, 60, 80, 64, 2, 2, 2, 12, 16, 20, 1, 1, 2, 16, 16, 64, 4, 1, 8, 2, 0]

Figure 5 also presents the PPAs of the five released official BOOMs [30] which are developed by senior engineers. These designs are not initially included in our design space $\mathcal{D}$. As Figure 5 shows, One-wide and Two-wide BOOM [11] are dominated by the Pareto-optimal set found by GRL-DSE and the other three BOOMs are new non-dominated Pareto-optimal. The microarchitectural parameters of our two Pareto-optimal designs are listed in TABLE V.

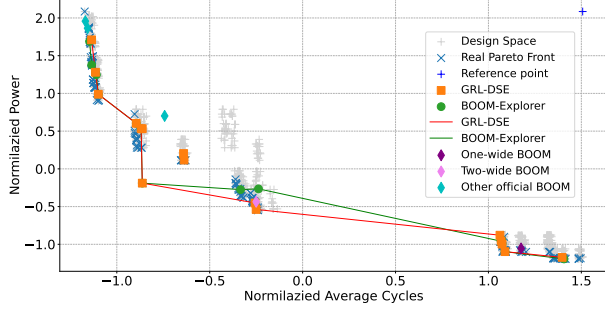
Evaluation results of the One-wide, Two-wide BOOM, and our two Pareto-optimal designs are shown in Table VI and TABLE VII, which show that GRL-DSE can efficiently find designs that outperform manual designs by 0.11%-3.26% in terms of PPA. Specifically, our Pareto design 1 speeds up benchmark mm, dhrystone, and spmv over 5% than the One-wide BOOM. If a larger design space is introduced for GRL-DSE, it should gain a larger Pareto-optimal set, striking better PPA trade-offs for a wide range of applications.

V. CONCLUSION

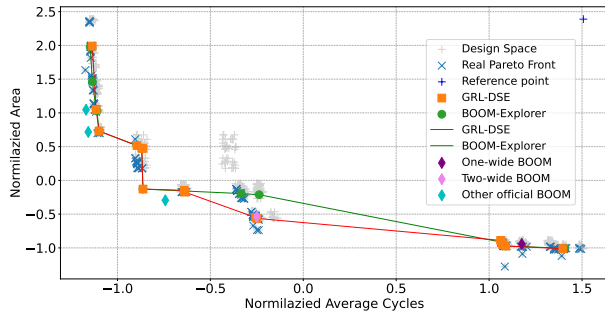
This paper focuses on the multi-objective design space exploration problem for modern microprocessors. We propose GRL-DSE, an automated microarchitecture search framework, which uses graph representation learning to capture microarchitecture knowledge and incorporate it into a compact design space, and then uses multi-objective Bayesian optimization with an ensemble surrogate model

TABLE VI: Comparison of performance (cycle number) between Pareto-optimal designs and manual designs on various benchmarks.

Benchmark	mm	median	mt-vvadd	mt-matmul	qsort	rsort	towers	vvadd	multiply	dhystone	spmv	pmp
<i>One-wide BOOM</i>	27037	7531	11479	27113	255811	176986	6354	2519	29899	210047	48755	159441
<i>Pareto Design 1</i>	24870	7515	11271	27094	255007	176052	6354	2523	29919	200040	45656	158015
<i>Speedup (%)</i>	8.713	0.213	1.845	0.070	0.315	0.531	0	-0.159	-0.067	5.002	6.788	0.902
<i>Two-wide BOOM</i>	16568	6747	10229	14108	223864	103419	3989	1349	20314	113110	30049	133992
<i>Pareto Design 2</i>	16568	6747	9947	14108	223263	104271	3989	1326	20314	113117	29620	133740
<i>Speedup (%)</i>	0	0	2.835	0	0.269	-0.817	0	1.735	0	-0.006	1.448	0.188



(a) Performance and power view of learned PPA Pareto front.



(b) Performance and area view of learned PPA Pareto front.

Fig. 5: Learned PPA Pareto front of BOOM microarchitectures. For clear presentation, the 3-D Pareto front is shown in two views, illustrated in Figure 4(b) as view 1 and view 2. Note that some design points are not Pareto-optimal at the 2-D view but are actually Pareto-optimal at 3-D PPA objective view. For proper hypervolume calculation, the reference point is the worst, i.e., the maximum, value of each dimension after normalization.

TABLE VII: Evaluation results comparison of Pareto design and manual design. IPC is the average number of instructions executed per cycle on the 12 evaluation benchmarks. Power unit: watts, area unit: μm^2 .

Microarchitecture	Average Cycles	IPC	Power	Area
<i>One-wide BOOM</i>	80247.667	0.714	2.871	32014.093
<i>Pareto Design 1</i>	78693.000	0.728	2.832	31002.941
<i>Improvement rate (%)</i>	1.94	1.98	1.38	3.26
<i>Two-wide BOOM</i>	56478.167	1.015	4.033	45211.880
<i>Pareto Design 2</i>	56417.500	1.016	4.002	44833.966
<i>Improvement rate (%)</i>	0.11	0.11	0.77	0.84

to conduct microarchitecture design space exploration. Using RISC-V BOOM as a testbed, experimental studies demonstrate that the proposed GRL-DSE outperforms the state-of-the-art in terms of Pareto-optimal quality, and the yield designs outperform manual designs developed by senior engineers. The proposed GRL-DSE also achieves the best performance compared with the winners at the ICCAD CAD Contest 2022.

ACKNOWLEDGEMENT

This work is supported partly by National Key R&D Program of China under Grant 2022YFB4400400, partly by National Natural Sci-

ence Foundation of China (NSFC) research project 62090025, partly by Science and Technology Commission of Shanghai Municipality Project (22DZ1101100).

REFERENCES

- [1] C. Bai and et al., "Boom-explorer: Risc-v boom microarchitecture design space exploration framework," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [2] E. İpek and et al., "Efficiently exploring architectural design spaces via predictive modeling," *ACM SIGOPS Operating Systems Review*, vol. 40, no. 5, pp. 195–206, 2006.
- [3] D. Li and et al., "Efficient design space exploration via statistical sampling and adaboost learning," in *2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 2016, pp. 1–6.
- [4] B. C. Lee and et al., "Illustrative design space studies with microarchitectural regression models," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. IEEE, 2007, pp. 340–351.
- [5] S. Khan and et al., "Using predictive modeling for cross-program design space exploration in multicore systems," in *16th International Conference on Parallel Architecture and Compilation Techniques (PACT 2007)*. IEEE, 2007, pp. 327–338.
- [6] P. Joseph and et al., "Construction and use of linear regression models for processor performance analysis," in *The Twelfth International Symposium on High-Performance Computer Architecture*, 2006. IEEE, 2006, pp. 99–108.
- [7] B. C. Lee and D. M. Brooks, "Accurate and efficient regression modeling for microarchitectural performance and power prediction," *ACM SIGOPS operating systems review*, vol. 40, no. 5, pp. 185–194, 2006.
- [8] P. Joseph, K. Vaswani, and et al., "A predictive performance model for super-scalar processors," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '06)*. IEEE, 2006, pp. 161–170.
- [9] G. Beltrame and et al., "Decision-theoretic design space exploration of multiprocessor platforms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 29, no. 7, pp. 1083–1095, 2010.
- [10] M. Zhang and et al., "D-vae: A variational autoencoder for directed acyclic graphs," *arXiv preprint arXiv:1904.11088*, 2019.
- [11] J. Zhao and et al., "Sonicboom: The 3rd generation berkeley out-of-order machine," in *Fourth Workshop on Computer Architecture Research with RISC-V*, vol. 5, 2020.
- [12] C. Celio and et al., "The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor," *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167*, 2015.
- [13] J. Bachrach and et al., "Chisel: constructing hardware in a scala embedded language," in *DAC Design automation conference 2012*. IEEE, 2012, pp. 1212–1221.
- [14] F. Scarselli and et al., "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.
- [15] K. Cho and et al., "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.
- [16] T. N. Kipf and et al., "Variational graph auto-encoders," *arXiv preprint arXiv:1611.07308*, 2016.
- [17] B. Shahriari and et al., "Taking the human out of the loop: A review of bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2015.
- [18] Q. Guo and et al., "Robust design space modeling," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 20, no. 2, pp. 1–22, 2015.
- [19] T. G. Dietterich and et al., "Ensemble learning," *The handbook of brain theory and neural networks*, vol. 2, no. 1, pp. 110–125, 2002.
- [20] C. E. Rasmussen and et al., "Gaussian processes in machine learning," in *Summer school on machine learning*. Springer, 2003, pp. 63–71.
- [21] B. Lakshminarayanan and et al., "Simple and scalable predictive uncertainty estimation using deep ensembles," *Advances in neural information processing systems*, vol. 30, 2017.
- [22] L. Breiman and et al., "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [23] S. Daulton and et al., "Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9851–9864, 2020.
- [24] S. Daulton, M. Balandat, and et al., "Parallel bayesian optimization of multiple noisy objectives with expected hypervolume improvement," *Advances in Neural Information Processing Systems*, vol. 34, pp. 2187–2200, 2021.
- [25] L. T. Clark and et al., "Asap7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [26] A. Amid and et al., "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [27] D. A. Van Veldhuizen and et al., *Multiobjective evolutionary algorithms: classifications, analyses, and new innovations*. Air Force Institute of Technology, 1999.
- [28] S. Li and et al., "Iccad cad contest 2022," 2022.
- [29] W. Jin and et al., "Junction tree variational autoencoder for molecular graph generation," in *International conference on machine learning*. PMLR, 2018, pp. 2323–2332.
- [30] J. Zhai and et al., "Mcpat-calib: A risc-v boom microarchitecture power modeling framework," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.